Partie 3 – Correction Gamma

Avant d'aborder la suite il nous faut discuter d'un point très important: la **colorimétrie**.

Les couleurs que vous visualisez sur un moniteur sont dites "perceptuelles" c'est-à-dire que le moniteur restitue une "réponse" en termes de signaux qui correspondent au triplet RGB.

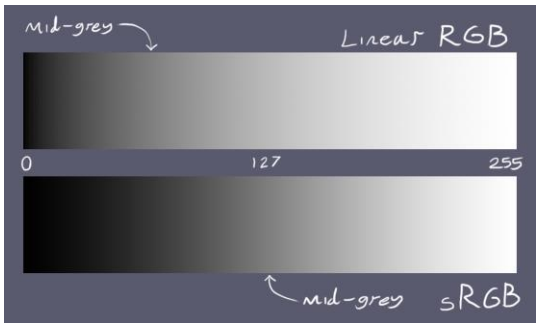
Mais en réalité cet aspect perceptuel tient à la fois du fonctionnement interne des convertisseurs du moniteur (la façon dont le courant est transformé en intensité lumineuse) et à la perceptuelle visuelle humaine.

Premièrement, la perception visuelle humaine est plus à même de distinguer les variations d'intensité des couleurs très intenses (blancs) et très sombres (noirs) mais moins sensible dans les gris.

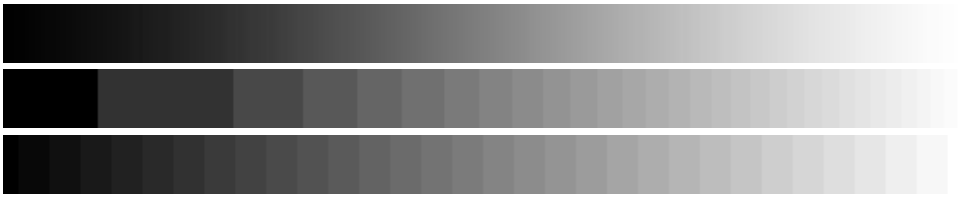
Secondement, le moniteur converti le courant en intensité à travers une loi de puissance (pow). Le facteur de puissance est appelé "gamma".
La plupart des moniteurs ont un gamma qui varie de l'ordre de 1.8 à 2.4. Il existe cependant une norme définie par Microsoft & HP nommée "sRGB" fixant le gamma approximativement à 2.2.

Tout ceci signifie deux choses: les moniteurs n'ont pas une réponse linéaire, c'est-à-dire que l'intensité croit en suivant une courbe de puissance (pow), et, cette courbe de puissance distribue plus de valeurs aux extrémités ce qui permet d'avoir des gradients plus lisses aux intensités minimales et maximales.

Pour prendre un exemple, lorsque vous souhaitez afficher un gris moyen à l'écran (127,127,127) - en OpenGL avec un appel à glClearColor(0.5f, 0.5f, 0.5, 1.f) - il s'agit d'un gris moyen dans le repère du moniteur, donc avec une correction (décompression) gamma. En linéaire ce gris moyen est plutôt un gris foncé (186,186,186)!



Prenons un autre exemple cette fois-ci afin de comprendre la finalité de la compression gamma. Supposons un gradient noir vers blanc que nous souhaitons afficher sur un écran ayant une très faible plage dynamique (32 couleurs, sur 5 bits donc), voici ce que cela donnerait :



gradient originel

gradient sur 5 bits

Compression gamma

C’est donc tout à la fois une bonne distribution des couleurs compte tenu de la faible dynamique (Low Dynamic Range – LDR) des moniteurs, et cela colle approximativement avec la vision humaine.

Cependant, cela pose un problème au programmeur : les opérations mathématiques -additions, soustractions, multiplications etc...- sont des opérations linéaires, mais les opérandes (les couleurs des pixels) ne sont pas linéaires (RGB) mais compressés gamma, on dira non-linéaire.

Ce que cela implique d’un point de vue colorimétrique :

- Le moniteur applique toujours une **décompression gamma** (mise à la puissance gamma)
- Une couleur visible sur un moniteur est donc toujours **non-linéaire**, car elle est exprimée dans le repère colorimétrique du moniteur.
- La plupart des formats d’image (PNG, JPEG etc...) stockent les images avec une **compression gamma** (donc non-linéaire également), majoritairement en sRGB 2.2.
- Une couleur compressée gamma (non-linéaire) traitée par un moniteur devient alors **linéaire** !

Il est alors nécessaire de se poser plusieurs questions lorsque l’on lit ou écrit des couleurs dans un shader.

(a) Si la couleur est linéaire on peut l’utiliser telle qu’elle dans nos opérations mathématiques. Autrement il faut la “linéariser” en appliquant une correction gamma.

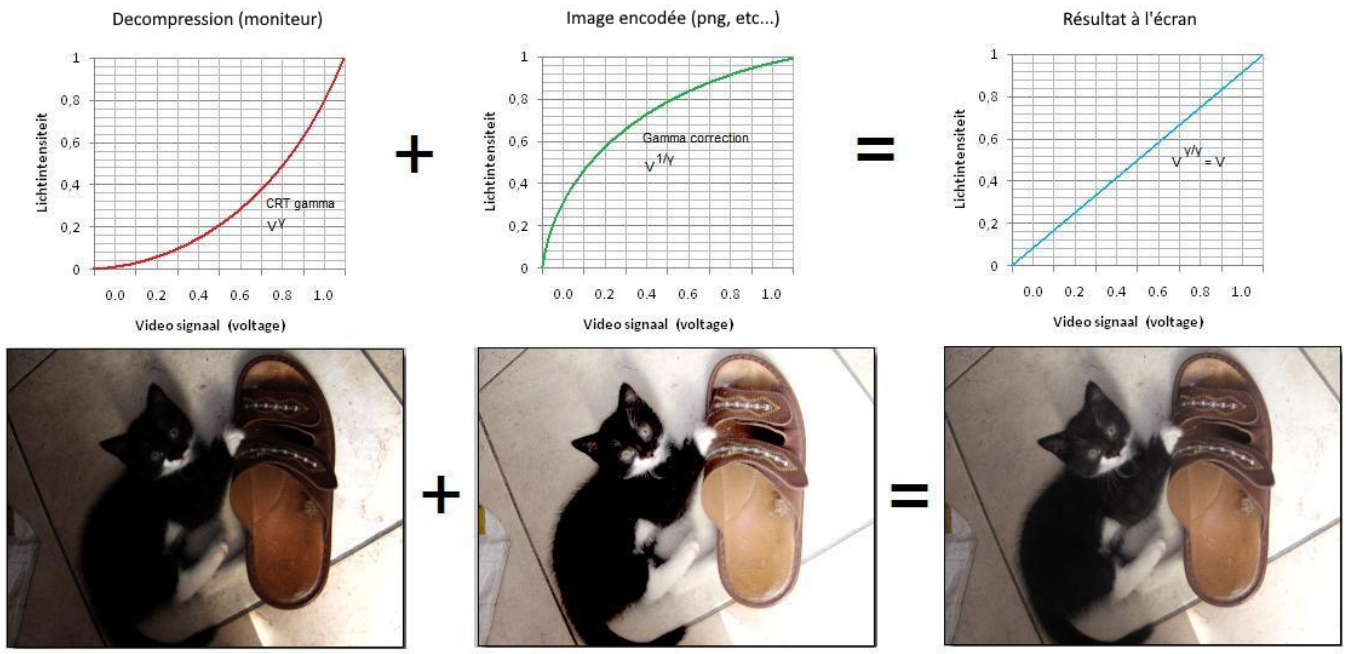
Couleur_SRC_non-lineaire = couleur_SRC$^{1./2.2}$ afin de corriger (linéariser) la couleur SRC il suffit d’appliquer la fonction inverse soit $x^{2.2}$, ce qui donne

Couleur_SRC_non-lineaire$^{2.2}$ = (Couleur_SRC$^{1./2.2}$)$^{2.2}$ = Couleur_SRC !

Comment savoir si une couleur en entrée est linéaire ? Il faut se poser la question de l’édition de la couleur. Est-ce que la couleur a été dessinée (image) ou capturée (color picker) ?
Si c’est le cas la couleur en entrée est non-linéaire.
Si au contraire la couleur a été générée par une équation (comme pour les normal maps, occlusion maps etc …) alors les données en entrée sont déjà linéaires.

(b) Comme le moniteur applique automatiquement une correction gamma il faut donc s’assurer que les données que l’on va écrire dans le color buffer correspondent aux attentes du moniteur.

Ainsi, si la couleur en sortie du shader est non-linéaire (correction gamma 1./2.2, sRGB) tout est ok. Cependant si la couleur en sortie est linéaire il faut lui appliquer une compression gamma de 1./2.2.



Exercice 3.1 appliquez les corrections gamma nécessaires en lecture et en écriture dans le Fragment Shader.

Heureusement pour nous, particulièrement pour les color buffers (y compris les textures) les GPUs sont capable d'effectuer pour nous ces conversions (à l'exception des couleurs passées en uniform ou attribut).

Ainsi on peut forcer la conversion automatique d'une texture lors de son échantillonnage en indiquant pour format interne un format sRGB comme **GL_SRGB8** ou **GL_SRGB8_ALPHA8**.
Notez que ALPHA8 est séparé de SRGB8 ce qui indique que la composante Alpha est toujours linéaire.

En sortie, on peut forcer les écritures dans le color buffer a convertir automatiquement de linéaire vers gamma à l'aide de la fonction **`glEnable(GL_FRAMEBUFFER_SRGB)`**;

Exercice 3.2 remplacez les changements précédents en appliquant les fonctions OpenGL aux textures ainsi qu'au Framebuffer.

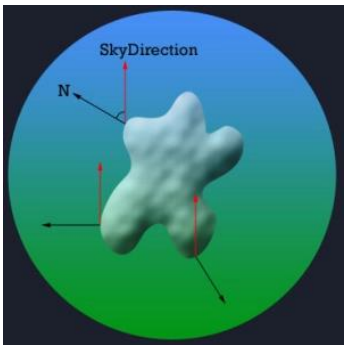
Références

https://www.cambridgeincolour.com/tutorials/gamma-correction.htm
https://medium.com/@Jacob_Bell/programmers-guide-to-gamma-correction-4c1d3a1724fb
https://gamedevelopment.tutsplus.com/articles/gamma-correction-and-why-it-matters--gamedev-14466
https://learnopengl.com/Advanced-Lighting/Gamma-Correction

2. Illumination hémisphèrique ambiante (hemispherical ambient illumination)

Dans le TP précédent nous avons implémenté les bases du modèle d'illumination de Phong.

Nous allons ajouter un facteur d'ambiance qui, plutôt que d'être une couleur unie, va combiner deux couleurs, une pour l'hémiphère "up" (le ciel) et l'autre pour l'hémisphère "down" (le sol).



La technique repose sur l'utilisation d'un vecteur de référence ("*SkyDirection*" dans l'image, par exemple (0, 1, 0)) et l'utilisation du produit scalaire.

En fonction du signe du produit scalaire entre la normale du fragment et le vecteur de référence, on va choisir la couleur du ciel ou du sol comme valeur ambiante.

Mais on peut faire mieux: on va mélanger les deux couleurs en fonction de la valeur du produit scalaire en l'interprétant comme un pourcentage.

Cependant, le produit scalaire est défini entre [-1,+1] lorsque les vecteurs sont normalisés.
Pour nous faciliter le mixage des couleurs on va re-parametrer le résultat produit scalaire de sorte à ce que le domaine soit [0,1].

Pour cela il suffit d'appliquer l'équation suivante : y = (x + 1) / 2 = x * ½ + ½ , ce qui nous donne :

```
HemisphereFactor = NdotSky * 0.5 + 0.5;
```

On utilise ensuite la fonction **mix**() du GLSL qui permet d'interpoler linéairement deux opérandes. On peut ainsi mixer deux couleurs SkyColor et GroundColor de façon paramétrique avec AmbientFactor.

```
vec3 AmbientColor = Ia * mix(SkyColor, GroundColor, HemisphereFactor);
...
vec3 FinalColor = AmbientColor + DiffuseColor + SpecularColor;
```

Références

https://thebookofshaders.com/glossary/?search=mix
Unity implémente cette technique sous la forme du mode TriLight qui ajoute une troisième couleur pour l'équateur : https://docs.unity3d.com/ScriptReference/Rendering.AmbientMode.Trilight.html